

Read-coherence

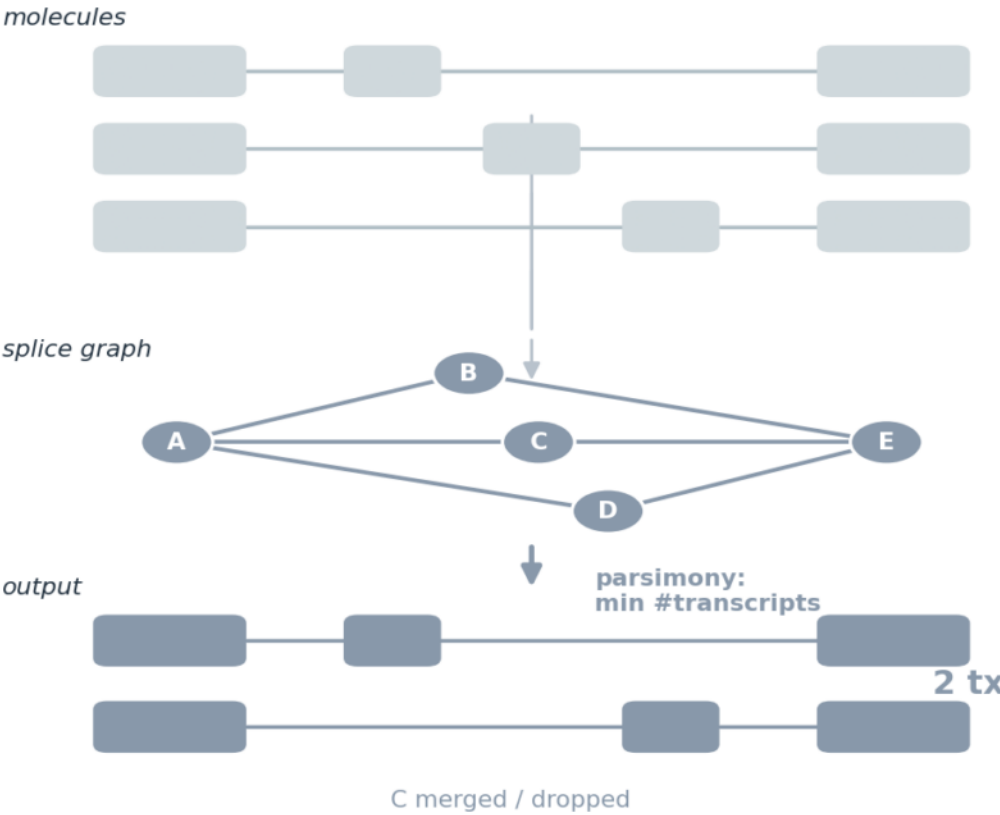
Per-molecule isoform recovery, additive over flow assembly

Recovers real isoforms StringTie's parsimony merges away —
as a provable superset of the flow output

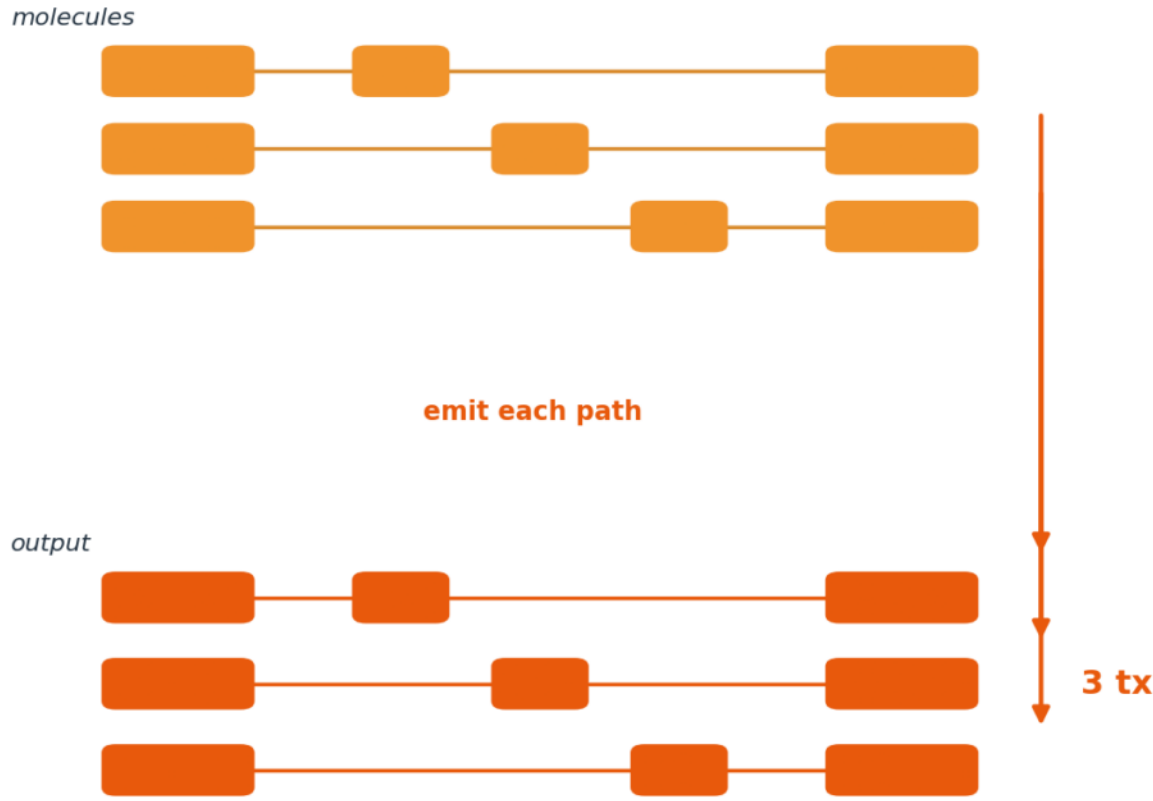
rustle · long-read transcript assembly

Why the two regimes differ

flow assembly (StringTie)

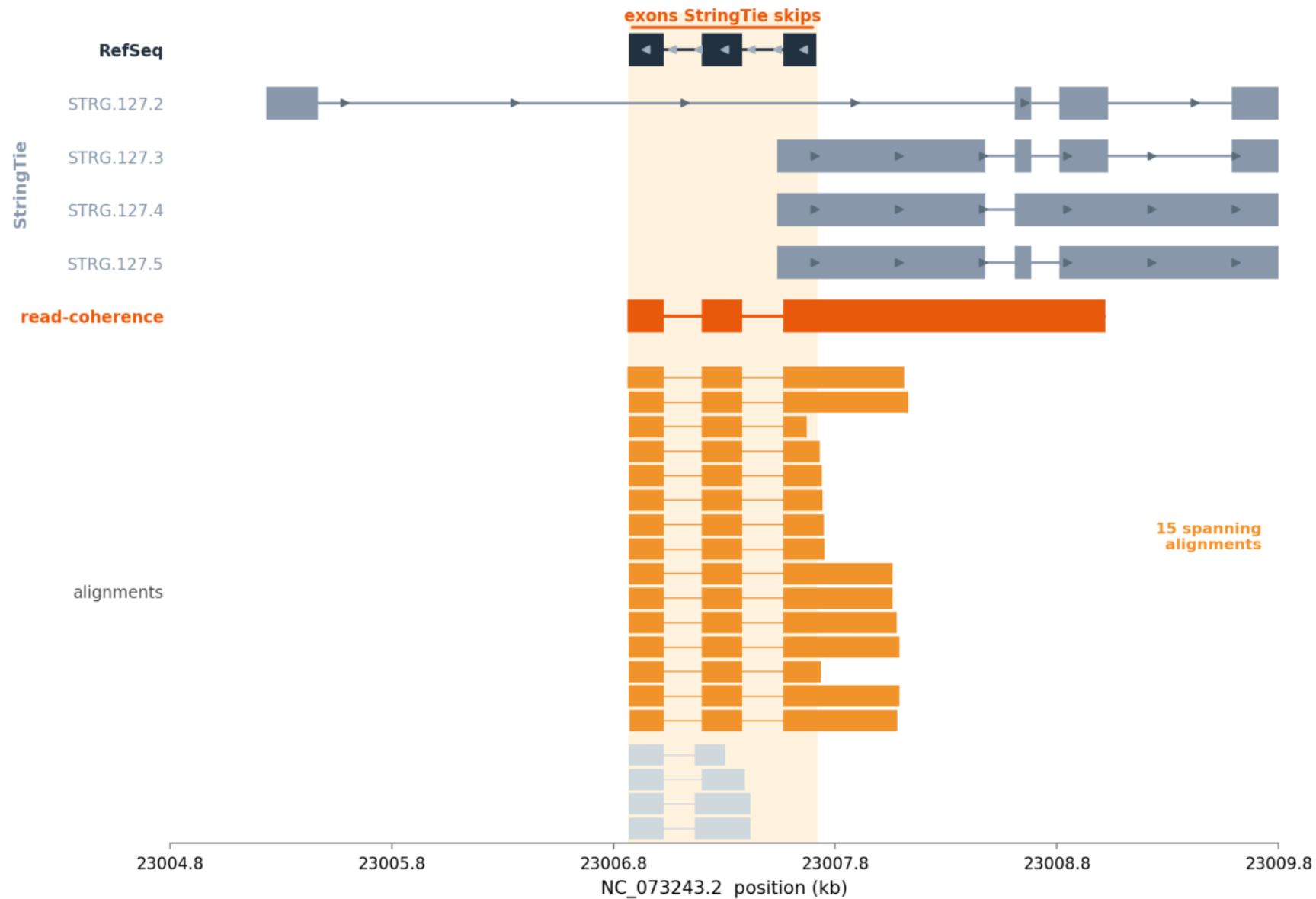


read-coherent extraction



□ molecule (alignments) ■ flow / StringTie ■ read-coherence

Read-coherence recovers a read-supported isoform StringTie skips



Identity = GROUP BY the exact junction string (not clustering)

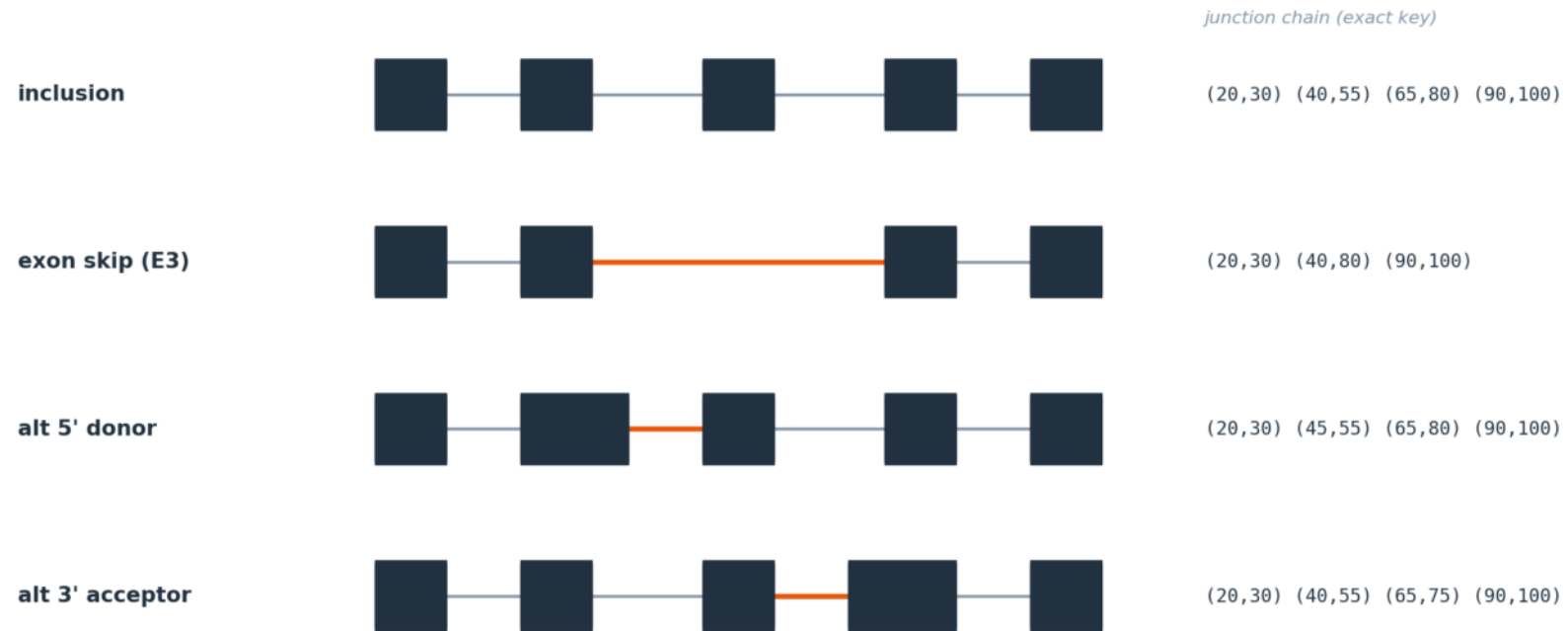
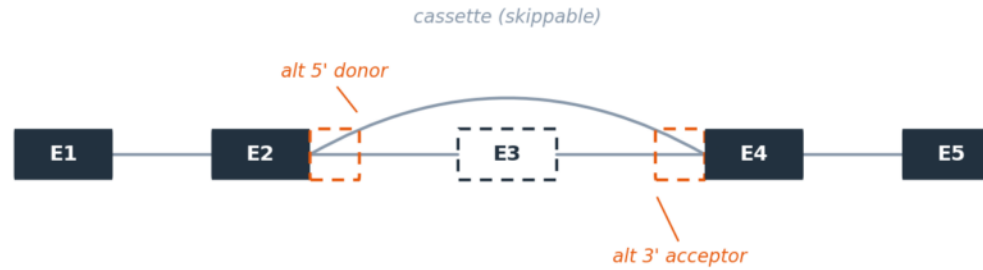
How a transcript is defined

- Read → its ordered list of splice junctions (introns)
- Same exact list → same transcript (hash key / GROUP BY)
- An equivalence relation: one unique, deterministic partition
- Truncated fragments → folded into their containing chain (sub-path)

Why it isn't clustering

- No distance, no similarity threshold, no merge radius
- No objective to optimize → result can't drift with a parameter
- Distinct junctions never merge (an exon-skip is not its inclusion isoform)
- Junctions matched exactly; small tolerance only at the 5'/3' ends

Alternative splicing = distinct junction chains

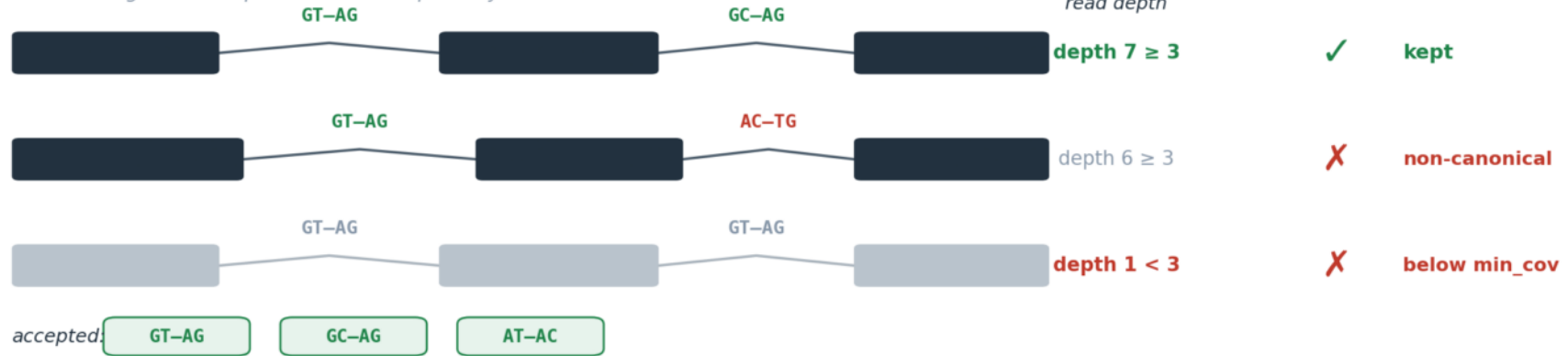


each event changes a junction → a different exact key → a distinct transcript

Two precision mechanisms

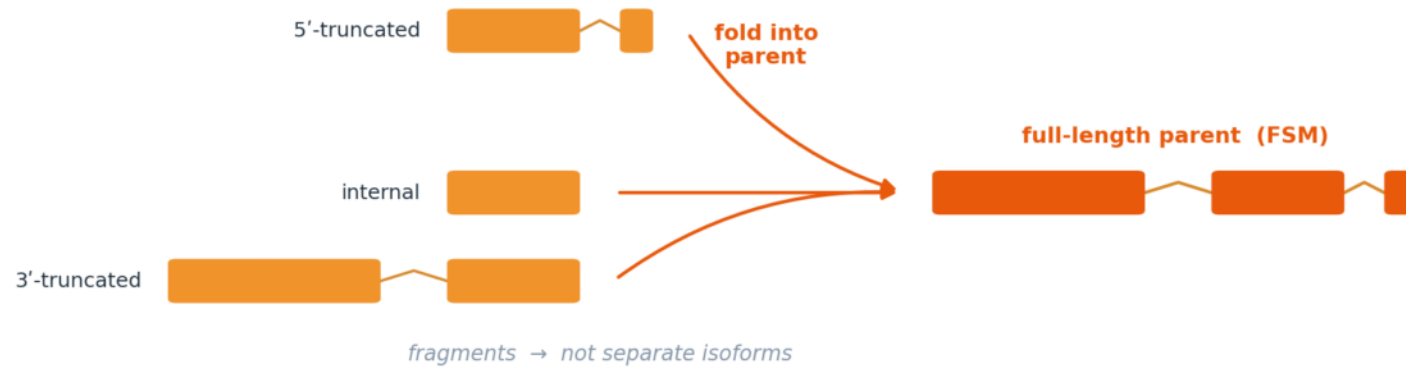
realness gate

annotation-free: genome sequence + read depth only



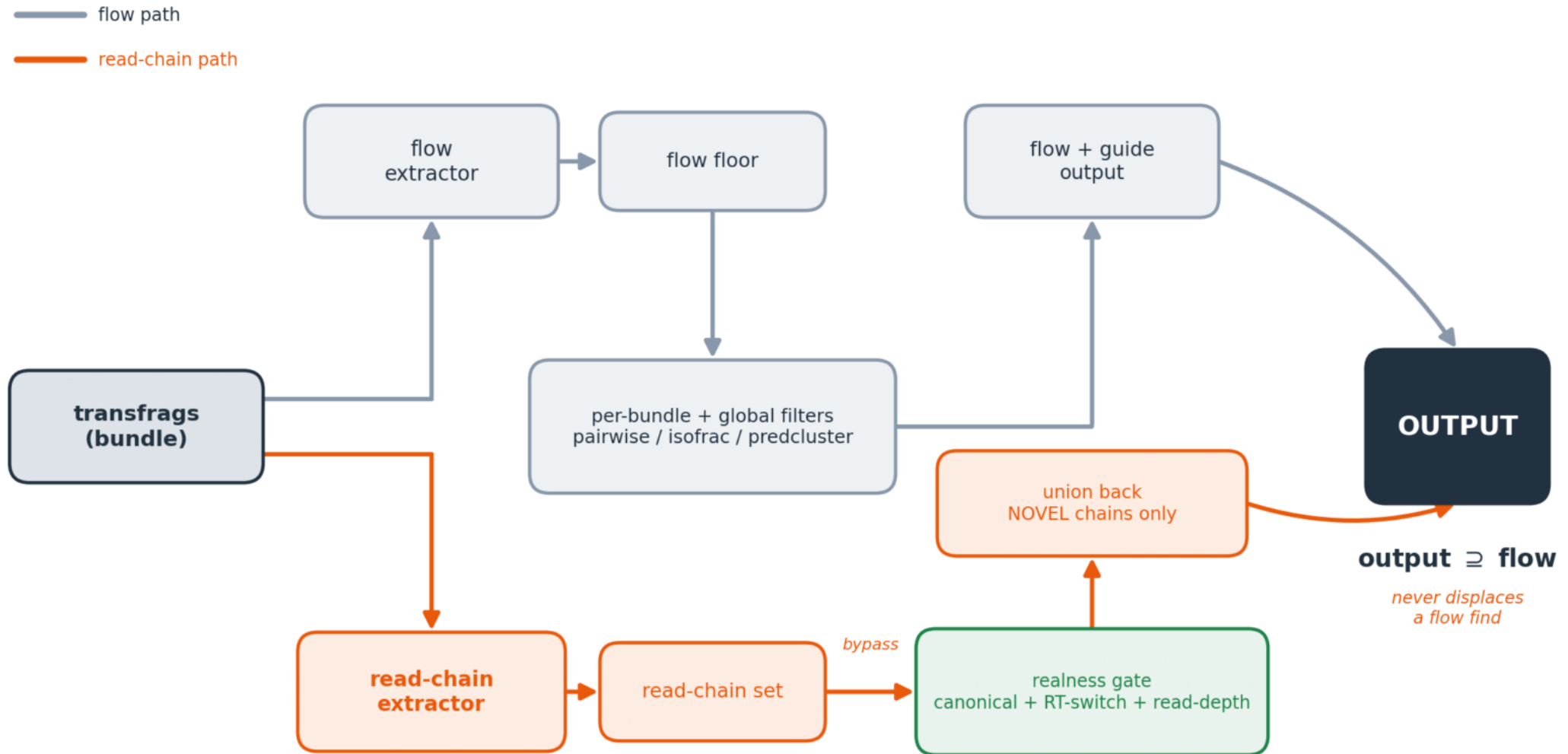
degradation-aware collapse

fragments fold into their full-length parent (ISM → FSM)



■ candidate / parent exon ■ fragment exon ■ read-coherence ● canonical / pass ● non-canonical / drop

Additive architecture: read-chain only adds



Alignments -> transcripts (pseudocode)

1. group alignments by their EXACT splice-junction chain

```
groups = {}                                # key = ordered list of (donor, acceptor)
for aln in long_read_alignments:
    chain = introns(aln)                    # splice junctions, in order
    if chain == []: continue                # single-exon -> skip
    g = groups[chain]                       # GROUP BY exact chain (hash key, not similarity)
    g.depth += aln.weight
    g.start = min(g.start, aln.start)      # 5'/3' ends = outer envelope
    g.end = max(g.end, aln.end)
```

2. fold truncated fragments into their full-length parent

```
for B in groups:
    A = longest chain that CONTAINS B as a sub-path # terminus within tol
    if A: A.depth += B.depth ; drop B               # ISM -> FSM
```

3. emit one transcript per surviving chain

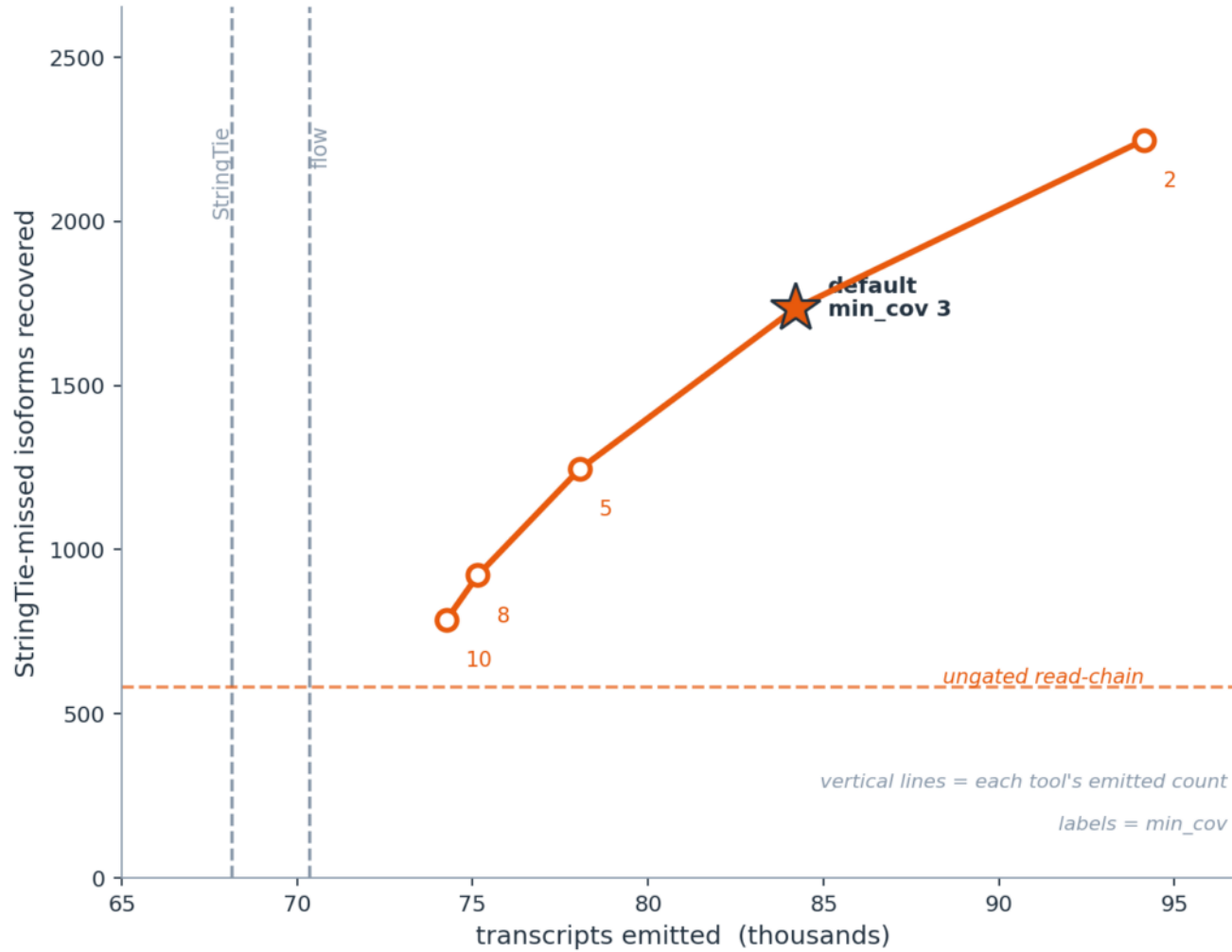
```
read_chains = [ Transcript(exons(chain, g.start, g.end), g.depth)
                 for chain, g in groups ]
```

4. add over the flow floor, gated (output >= flow)

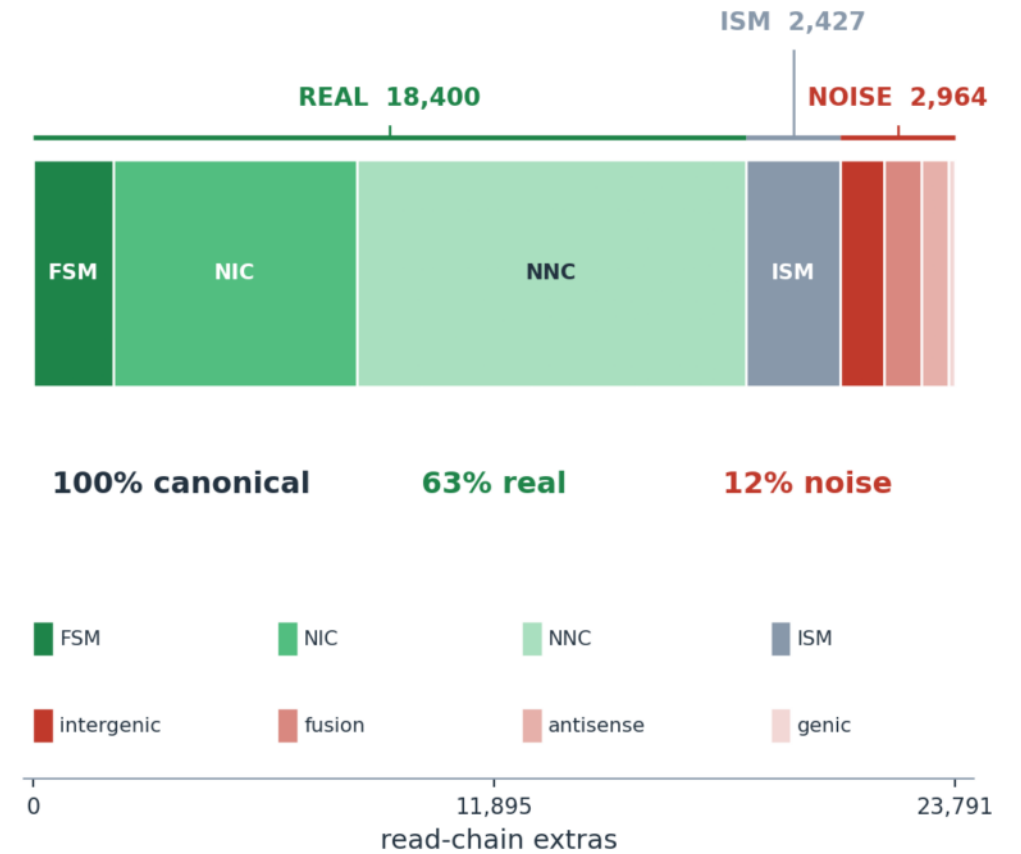
```
output = flow_assembly(reads)              # StringTie-style baseline
seen = { introns(t) for t in output }
for t in read_chains:                      # held out of the flow filters
    if introns(t) in seen: continue        # already there -> no-op
    if any junction not canonical(t): continue # GT-AG / GC-AG / AT-AC
    if rt_switch(t) or t.depth < K: continue # realness gate
    output.add(t)                          # novel + real -> add (never removes a flow tx)
```

steps 1-3 = the read-chain path in the previous slide; step 4 = hold-out + gate + union-back

Recall vs transcript count



What the extras are (SQANTI3, n=23,791)



Positioning: read-coherence is the established collapse paradigm

Prior art (per-molecule collapse)

- IsoSeq collapse / cDNA_Cupcake — collapse full-length reads by junction chains
- IsoQuant — per-read intron graph; already beats StringTie on novel isoforms (lower FP)
- FLAIR / TALON / Mandalorion — per-read isoforms by splice structure
- So "per-molecule > flow assembly on recall" is already known

What this adds

- A provably-additive layer over a StringTie flow port (output $\supseteq$ flow), in one tool
- Annotation-free realness gate (canonical incl. GC-AG/AT-AC + RT-switch + depth)
- The substrate for the novel part: paralog-copy resolution via PSVs →

IsoQuant, Nat Biotech 2023 · FLAIR2, Genome Biol 2024 · Paraphase, Nat Commun 2025

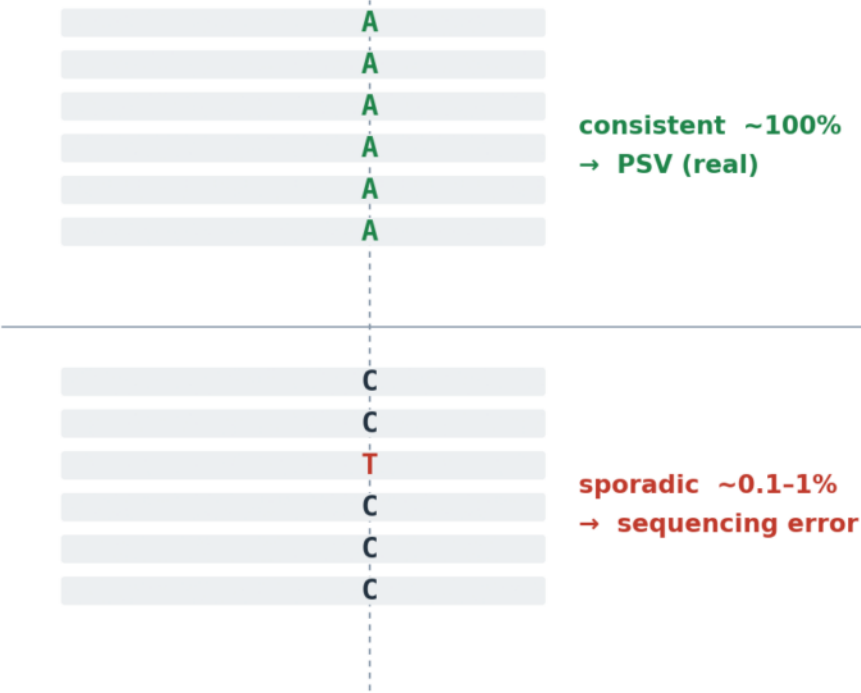
Calling a PSV: a fixed inter-copy column, confirmed across reads

1. align the copies (POA) → homologous columns

					PSV					PSV		
					I					I		
Copy A	G	C	A	T	A	C	G	T	G	T	A	C
Copy B	G	C	A	T	G	C	G	T	G	T	A	C
Copy C	G	C	A	T	A	C	G	T	G	C	A	C

PSV = column where copies' fixed bases differ

2. confirm across reads (PSV vs sequencing error)



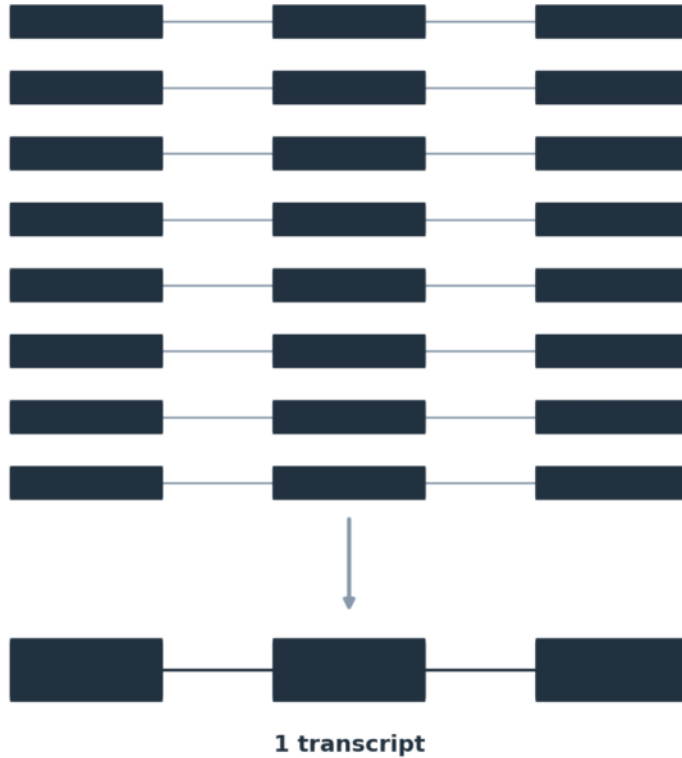
fixed & consistent across every read vs random / rare

identifiable iff $\geq K$ PSV columns co-segregate (K independent error-agreements are improbable)

read → copy by which PSV alleles it carries (+ NM / AS)

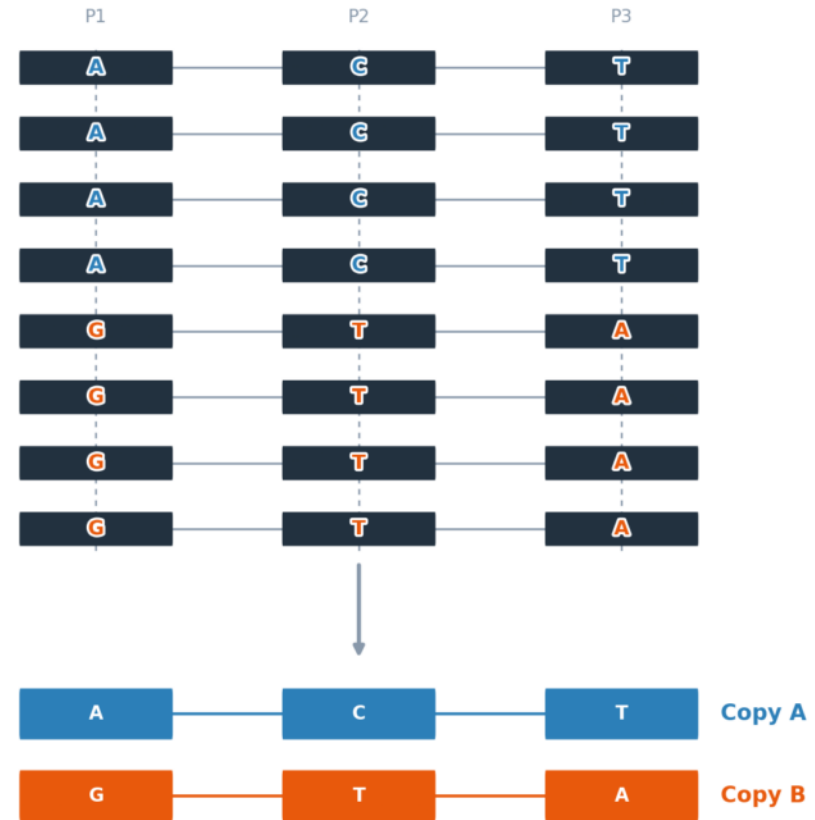
Using PSVs: same junction chain, split by allele into copies

read-coherence (junctions only)



near-identical paralogs share the junction chain
→ one bucket (copies merged)

read-coherence + PSVs



PSV-allele vector splits the bucket → two paralog copies

transcript key = junction chain (+) PSV-allele vector

structural axis

copy axis

PSV = base difference consistent across reads (not sequencing error)

Thesis contribution: the open cell

identity resolved

	— (none)	allele / haplotype	paralog copy
task			
quantify known transcripts	Salmon / kallisto (short-read)	RPVG · pantas (spliced pangenome)	Paraphase* <i>* DNA / gene level, not isoforms</i>
de-novo isoform assembly	IsoQuant · Cupcake FLAIR · LRAA	HapIso (diploid)	★ THIS THESIS <i>the open cell</i>

object: spliced family variation graph: a transcript = a path over junctions (+) PSV alleles; a copy = an allele-consistent haplotype + its isoforms

theorem: identifiable iff copies differ at $\geq K$ PSV columns spanned by enough reads (else provably merged)